

Arquivo: frame.py

```
"""
DESCRIÇÃO GERAL:
Este módulo implementa o conceito de "Framing" (Enquadramento).
O protocolo TCP é orientado a fluxo de bytes (stream), o que significa que ele não
tem o conceito natural de "início e fim" de uma mensagem. Se você envia duas mensagens
de 10 bytes, o destinatário pode ler 20 bytes de uma vez, ou 5 bytes de cada vez.
Para resolver isso, este módulo anexa um "cabeçalho" de 4 bytes antes de cada mensagem,
indicando o tamanho exato do payload (conteúdo) que virá a seguir.
"""

# Importa a biblioteca de sockets para comunicação de rede de baixo nível.
import socket

# struct converte valores do Python (como inteiros) em strings de bytes formatadas.
# Essencial para empacotar o tamanho da mensagem em um formato binário previsível (4 bytes).
import struct

def send_frame(sock: socket.socket, payload: bytes) -> None:
    """
    Envia uma mensagem pela rede usando framing por tamanho.

    Estrutura transmitida:
    [4 bytes de tamanho][payload binário]
    """
    # struct.pack serializa o tamanho do payload.
    # "!I" significa: "!" = Network Byte Order (Big-Endian, padrão de rede),
    # "I" = Unsigned Integer (inteiro sem sinal de 32 bits / 4 bytes).
    # Assim, garantimos que o tamanho ocupará exatamente 4 bytes.
    header = struct.pack("!I", len(payload))

    # sendall garante que todos os bytes (cabeçalho + payload) sejam enviados pela rede.
    # Diferente do sock.send(), que poderia enviar apenas uma parte se o buffer do SO estivesse cheio.
    sock.sendall(header + payload)

def recv_exact(sock: socket.socket, n: int) -> bytes:
    """
    Lê exatamente n bytes do socket.

    Isso é importante porque recv() pode retornar menos bytes do que o esperado.
    Se a conexão cair no meio da leitura, uma exceção é lançada.
    """
    # Lista para acumular os pedaços (chunks) recebidos.
    chunks = []
    # Variável que controla quantos bytes ainda faltam ler.
    remaining = n

    # Loop continua até termos lido exatamente 'n' bytes.
    while remaining > 0:
        # Tenta ler a quantidade de bytes que ainda falta.
```

```

chunk = sock.recv(remaining)

# Se recv() retornar uma string de bytes vazia, significa que o outro
# lado fechou a conexão inesperadamente (EOF).
if not chunk:
    raise ConnectionError("Conexão encerrada antes do esperado")

# Adiciona o pedaço recebido à nossa lista.
chunks.append(chunk)
# Reduz o contador de bytes restantes.
remaining -= len(chunk)

# Junta todos os pedaços acumulados em uma única string de bytes e a retorna.
return b"".join(chunks)

def recv_frame(sock: socket.socket) -> bytes:
    """
    Lê uma mensagem completa enviada pelo protocolo de framing.
    Primeiro lê os 4 bytes do cabeçalho e depois lê o payload.
    """
    # Primeiro, obrigatoriamente lemos os primeiros 4 bytes (que contêm o tamanho do payload).
    header = recv_exact(sock, 4)

    # struct.unpack faz o inverso do pack: converte os 4 bytes binários de volta para um inteiro Python.
    # Como unpack retorna uma tupla (mesmo que haja só um valor), pegamos o índice [0].
    size = struct.unpack("!I", header)[0]

    # Agora que sabemos o tamanho exato da mensagem, usamos recv_exact novamente
    # para ler o payload inteiro com segurança, sem ler lixo ou bytes da próxima mensagem.
    return recv_exact(sock, size)

```